
Firewall Policy and DoS Mitigation

Zone-based firewall configuration

Jorge Mendes

Network security configuration

2 Firewalls

2.1 Exercise 5 - Protecting a campus network with ZBPF

2.1.1 Initial network configuration and addressing plan

The topology was built with four zones: PR1, PR2, DMZ, and OUT. The firewall was implemented on a Cisco 7200 router, and each zone received at least one test node to run `ping`, `nmap`, SSH, and service validation. The DMZ included three logical servers: Web, Email, and DNS.

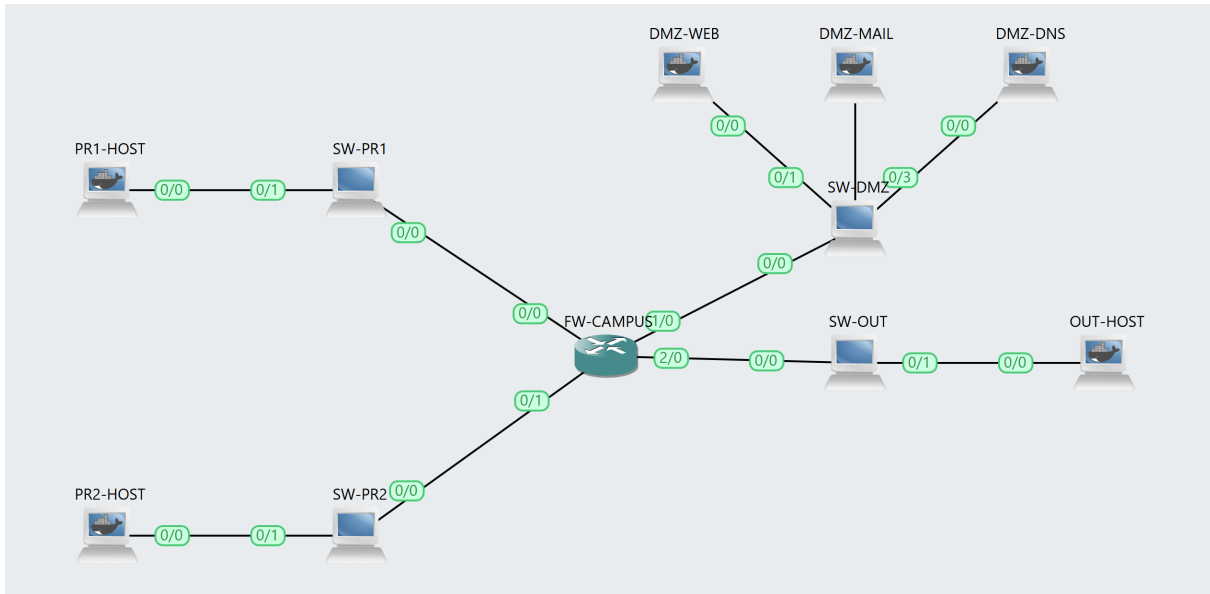


Figure 25: Campus network topology implemented in GNS3 (in GNS3 accessed through the browser, the switch icons appear with the default PC icon, even though they are switches).

Table 1: Addressing plan used in the campus network.

Zone	Network	Main equipment
PR1	10.1.1.0/24	PR1 host: 10.1.1.10; firewall: 10.1.1.254
PR2	10.2.2.0/24	PR2 host: 10.2.2.10; firewall: 10.2.2.254
DMZ	172.16.10.0/24	Web: 172.16.10.10; Email: 172.16.10.20; DNS: 172.16.10.30
OUT	203.0.113.0/24	OUT host: 203.0.113.10; firewall: 203.0.113.254

Before applying the ZBPF policies, it was confirmed that routing, services, and NAT were operational. Output 5 shows a NAT *overload* translation created for traffic originating in PR1 towards OUT. This confirms that private addresses from the 10.0.0.0/8 network were translated to the address of the firewall's OUT interface, as defined in the assignment.

Listing 5: Validation output for NAT and directly connected routes.

```
show ip nat translations

Pro Inside global Inside local Outside local Outside global
icmp 203.0.113.254:41 10.1.1.10:41 203.0.113.10:41 203.0.113.10:41
icmp 203.0.113.254:4 10.2.2.10:4 203.0.113.10:4 203.0.113.10:4

show ip route connected
```

```

C 10.1.1.0/24 is directly connected, FastEthernet0/0
L 10.1.1.254/32 is directly connected, FastEthernet0/0
C 10.2.2.0/24 is directly connected, FastEthernet0/1
L 10.2.2.254/32 is directly connected, FastEthernet0/1
C 172.16.10.0/24 is directly connected, FastEthernet1/0
L 172.16.10.254/32 is directly connected, FastEthernet1/0
C 203.0.113.0/24 is directly connected, FastEthernet2/0
L 203.0.113.254/32 is directly connected, FastEthernet2/0

```

2.1.2 DMZ services required by the assignment

The DMZ servers were prepared according to the specified policy: Web with HTTP and HTTPS, Email with SMTP, POP3, and IMAP, and DNS with service on port 53. The goal was not to test complete email applications, but to demonstrate that the firewall allows or blocks the correct ports. Output 6 summarizes the local validation of the services.

Listing 6: Output of local validation of DMZ services.

```

DMZ-WEB:
connect_ex 172.16.10.10:80 -> 0
connect_ex 172.16.10.10:443 -> 0

DMZ-MAIL:
connect_ex 172.16.10.20:25 -> 0
connect_ex 172.16.10.20:110 -> 0
connect_ex 172.16.10.20:143 -> 0

DMZ-DNS:
connect_ex 172.16.10.30:53 -> 0

```

2.1.3 ZBPF firewall configuration

The configuration was structured by traffic direction. The ACLs identify the flows allowed by each policy, the *class maps* group each ACL, and the *policy maps* apply **inspect** to allowed traffic and **drop log** to the remaining traffic. This choice is consistent with the theoretical ZBPF model: classify, apply an action, and associate the policy with a *zone pair* [2].

Output 7 shows the relevant ACLs. The absence of permissions from PR1 to PR2, PR2 to PR1, OUT to PR1, or DMZ to PR1 causes those flows to be blocked by ZBPF's *default drop* behavior and by the *default* classes of the policies.

Listing 7: Configuration of the main ACLs for the campus network.

```

ip access-list extended ACL_DMZ_TO_OUT
 permit icmp 172.16.10.0 0.0.0.255 any
 permit tcp 172.16.10.0 0.0.0.255 any eq smtp
 permit tcp 172.16.10.0 0.0.0.255 any eq domain
 permit udp 172.16.10.0 0.0.0.255 any eq domain

ip access-list extended ACL_OUT_TO_DMZ
 permit icmp any host 172.16.10.10
 permit icmp any host 172.16.10.20
 permit icmp any host 172.16.10.30
 permit tcp any host 172.16.10.10 eq www
 permit tcp any host 172.16.10.10 eq 443
 permit tcp any host 172.16.10.20 eq smtp
 permit tcp any host 172.16.10.30 eq domain
 permit udp any host 172.16.10.30 eq domain

```

```

ip access-list extended ACL_PR_TO_DMZ
  permit icmp 10.0.0.0 0.255.255.255 host 172.16.10.10
  permit icmp 10.0.0.0 0.255.255.255 host 172.16.10.20
  permit icmp 10.0.0.0 0.255.255.255 host 172.16.10.30
  permit tcp 10.0.0.0 0.255.255.255 host 172.16.10.10 eq www
  permit tcp 10.0.0.0 0.255.255.255 host 172.16.10.10 eq 443
  permit tcp 10.0.0.0 0.255.255.255 host 172.16.10.20 eq smtp
  permit tcp 10.0.0.0 0.255.255.255 host 172.16.10.20 eq pop3
  permit tcp 10.0.0.0 0.255.255.255 host 172.16.10.20 eq 143
  permit tcp 10.0.0.0 0.255.255.255 host 172.16.10.30 eq domain
  permit udp 10.0.0.0 0.255.255.255 host 172.16.10.30 eq domain

ip access-list extended ACL_PR_TO_OUT
  permit icmp 10.0.0.0 0.255.255.255 any
  permit tcp 10.0.0.0 0.255.255.255 any eq www
  permit tcp 10.0.0.0 0.255.255.255 any eq 443
  permit tcp 10.0.0.0 0.255.255.255 any eq domain
  permit udp 10.0.0.0 0.255.255.255 any eq domain

ip access-list extended ACL_TO_SELF_SSH
  permit tcp any any eq 22

```

Listing 8: Class maps and policy maps of the ZBPF configuration.

```

class-map type inspect match-any CM_PR_TO_OUT
  match access-group name ACL_PR_TO_OUT
class-map type inspect match-any CM_PR_TO_DMZ
  match access-group name ACL_PR_TO_DMZ
class-map type inspect match-any CM_TO_SELF_SSH
  match access-group name ACL_TO_SELF_SSH
class-map type inspect match-any CM_OUT_TO_DMZ
  match access-group name ACL_OUT_TO_DMZ
class-map type inspect match-any CM_DMZ_TO_OUT
  match access-group name ACL_DMZ_TO_OUT

policy-map type inspect PM_PR_TO_DMZ
  class type inspect CM_PR_TO_DMZ
    inspect
  class class-default
    drop log

policy-map type inspect PM_PR_TO_OUT
  class type inspect CM_PR_TO_OUT
    inspect
  class class-default
    drop log

policy-map type inspect PM_TO_SELF
  class type inspect CM_TO_SELF_SSH
    inspect
  class class-default
    drop log

policy-map type inspect PM_OUT_TO_DMZ
  class type inspect CM_OUT_TO_DMZ
    inspect
  class class-default
    drop log

policy-map type inspect PM_DMZ_TO_OUT
  class type inspect CM_DMZ_TO_OUT
    inspect
  class class-default

```

```
drop log
```

To comply with the modularity note in the assignment, the DMZ policy was later refined with nested *class maps*. In this version, protocols are grouped into dedicated classes, DMZ destinations are filtered by specific ACLs, and the final classes combine destination and protocol with **match class-map**. Output 9 shows the structure applied from PR/OUT to the DMZ. The resulting policy kept the same behavior observed in the tests, but now followed the modular strategy suggested in the lab sheet.

Listing 9: Modular configuration with nested class maps for access to the DMZ.

```
ip access-list extended ACL_DST_WEB
  permit ip any host 172.16.10.10
ip access-list extended ACL_DST_MAIL
  permit ip any host 172.16.10.20
ip access-list extended ACL_DST_DNS
  permit ip any host 172.16.10.30
ip access-list extended ACL_PROTO_WEB
  permit tcp any any eq www
  permit tcp any any eq 443
ip access-list extended ACL_PROTO_MAIL_FULL
  permit tcp any any eq smtp
  permit tcp any any eq pop3
  permit tcp any any eq 143
ip access-list extended ACL_PROTO_DNS
  permit tcp any any eq domain
  permit udp any any eq domain
ip access-list extended ACL_PROTO_SMTP
  permit tcp any any eq smtp
ip access-list extended ACL_PROTO_ICMP
  permit icmp any any
ip access-list extended ACL_DST_DMZ_SERVERS
  permit ip any host 172.16.10.10
  permit ip any host 172.16.10.20
  permit ip any host 172.16.10.30

class-map type inspect match-any CM_PROTO_WEB
  match access-group name ACL_PROTO_WEB
class-map type inspect match-any CM_PROTO_MAIL_FULL
  match access-group name ACL_PROTO_MAIL_FULL
class-map type inspect match-any CM_PROTO_SMTP
  match access-group name ACL_PROTO_SMTP
class-map type inspect match-any CM_PROTO_DNS
  match access-group name ACL_PROTO_DNS
class-map type inspect match-any CM_PROTO_ICMP
  match access-group name ACL_PROTO_ICMP

class-map type inspect match-all CM_TO_WEB_NESTED
  match access-group name ACL_DST_WEB
  match class-map CM_PROTO_WEB
class-map type inspect match-all CM_TO_MAIL_FULL_NESTED
  match access-group name ACL_DST_MAIL
  match class-map CM_PROTO_MAIL_FULL
class-map type inspect match-all CM_TO_MAIL_SMTP_NESTED
  match access-group name ACL_DST_MAIL
  match class-map CM_PROTO_SMTP
class-map type inspect match-all CM_TO_DNS_NESTED
  match access-group name ACL_DST_DNS
  match class-map CM_PROTO_DNS
class-map type inspect match-all CM_TO_DMZ_ICMP_NESTED
  match access-group name ACL_DST_DMZ_SERVERS
  match class-map CM_PROTO_ICMP
```

```

class-map type inspect match-any CM_PR_TO_DMZ_NESTED
  match class-map CM_TO_WEB_NESTED
  match class-map CM_TO_MAIL_FULL_NESTED
  match class-map CM_TO_DNS_NESTED
  match class-map CM_TO_DMZ_ICMP_NESTED

class-map type inspect match-any CM_OUT_TO_DMZ_NESTED
  match class-map CM_TO_WEB_NESTED
  match class-map CM_TO_MAIL_SMTP_NESTED
  match class-map CM_TO_DNS_NESTED
  match class-map CM_TO_DMZ_ICMP_NESTED

policy-map type inspect PM_PR_TO_DMZ
  class type inspect CM_PR_TO_DMZ_NESTED
  inspect
  class class-default
  drop log

policy-map type inspect PM_OUT_TO_DMZ
  class type inspect CM_OUT_TO_DMZ_NESTED
  inspect
  class class-default
  drop log

```

The *zone pairs* were created only for the directions allowed by the assignment. No *zone pair* was created between PR1 and PR2, or from external zones to private zones. Thus, the resulting policy blocks communication between private networks and prevents access to the private networks from other zones.

Listing 10: Zone pairs applied on the campus firewall.

```

zone-pair security PR1_TO_OUT source PR1 destination OUT
  service-policy type inspect PM_PR_TO_OUT
zone-pair security PR2_TO_OUT source PR2 destination OUT
  service-policy type inspect PM_PR_TO_OUT
zone-pair security PR1_TO_DMZ source PR1 destination DMZ
  service-policy type inspect PM_PR_TO_DMZ
zone-pair security PR2_TO_DMZ source PR2 destination DMZ
  service-policy type inspect PM_PR_TO_DMZ
zone-pair security OUT_TO_DMZ source OUT destination DMZ
  service-policy type inspect PM_OUT_TO_DMZ
zone-pair security DMZ_TO_OUT source DMZ destination OUT
  service-policy type inspect PM_DMZ_TO_OUT
zone-pair security PR1_TO_SELF source PR1 destination self
  service-policy type inspect PM_TO_SELF
zone-pair security PR2_TO_SELF source PR2 destination self
  service-policy type inspect PM_TO_SELF
zone-pair security DMZ_TO_SELF source DMZ destination self
  service-policy type inspect PM_TO_SELF
zone-pair security OUT_TO_SELF source OUT destination self
  service-policy type inspect PM_TO_SELF

```

2.1.4 Validation of the requirements

Table 2 summarizes the validation of the ten requirements of the campus policy:

Table 2: Validation of the campus policy in the order of the assignment.

Req.	Required policy	Observed evidence
1	PR1 and PR2 cannot communicate with each other.	<code>ping</code> PR1 → PR2 and PR2 → PR1 with 100% loss in Outputs 11 and 12.
2	The private zones use 10.0.0.0/8 and NAT <i>overload</i> to OUT.	Translations of 10.1.1.10 and 10.2.2.10 to 203.0.113.254 in Output 5.
3	The private zones cannot be accessed from other zones.	OUT → PR1 and DMZ → PR1 with 100% loss in Outputs 14 and 15.
4	The DMZ contains Web, Email, and DNS servers accessible from OUT, PR1, and PR2.	Local services in Output 6; allowed access in Outputs 11, 12, and 14.
5	PR1 and PR2 to OUT only with HTTP, HTTPS, ICMP, and DNS.	ICMP, HTTP, HTTPS, and DNS validated from PR1 and PR2; SSH/SMTP blocked in Output 13.
6	PR1 and PR2 to the DMZ only for Web, ICMP, Email, and DNS on the three servers.	NMAP from PR1 and PR2 shows allowed ports as open and disallowed ports as filtered in Outputs 11 and 12.
7	DMZ to OUT only with ICMP, SMTP, and DNS.	ICMP allowed; SMTP/DNS reach the host as closed or open filtered ; HTTP remains filtered in Outputs 15 and 16.
8	OUT to DMZ only with Web, ICMP, SMTP, and DNS.	HTTP/HTTPS on Web, SMTP on Email, and DNS TCP/UDP on DNS are accessible; POP3/IMAP/SSH remain filtered in Outputs 14 and 16.
9	The firewall can only be accessed by SSH from all zones.	Port 22 open and Telnet filtered in PR1, PR2, DMZ, and OUT in Output 18; real SSH session in Output 17.
10	Everything that is not allowed must be denied.	Unauthorized ports appear as filtered ; the <i>policy maps</i> end with <code>class-default drop log</code> in Output 8.

2.1.5 Requirement 1 - PR1 and PR2 do not communicate; requirements 5 and 6 - authorized outbound traffic from the private zones

The policy required that PR1 and PR2 could not communicate with each other, that the private zones could reach OUT only with HTTP, HTTPS, ICMP, and DNS, and that they could access the DMZ only on the permitted services. Output 11 shows these results from PR1.

Listing 11: NMAP and ICMP tests from PR1.

```
ping -c 2 -W 2 10.2.2.10
--- 10.2.2.10 ping statistics ---
```

```
2 packets transmitted, 0 received, 100% packet loss
```

```
ping -c 2 -W 2 203.0.113.10
--- 203.0.113.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
```

```
nmap -n -Pn -p 80,443,22 172.16.10.10
PORT STATE SERVICE
22/tcp filtered ssh
80/tcp open http
443/tcp open https
```

```
nmap -n -Pn -p 25,110,143,80 172.16.10.20
PORT STATE SERVICE
25/tcp open smtp
80/tcp filtered http
110/tcp open pop3
143/tcp open imap
```

```
nmap -n -Pn -p 53,25 172.16.10.30
PORT STATE SERVICE
25/tcp filtered smtp
53/tcp open domain
```

The result confirms that PR1 does not communicate with PR2, since the ping to 10.2.2.10 had 100% loss. It also confirms that PR1 can reach OUT via ICMP. On the DMZ servers, the expected ports appear as **open**, while unauthorized ports appear as **filtered**. In NMAP, **filtered** means that the packet was blocked by a filter, in this case the firewall, while **closed** would indicate that the packet reached the host but the service was not open [3].

Output 12 repeats the validation from PR2: PR1 is unreachable, OUT responds via ICMP, the permitted services in the DMZ are accessible, and DNS is confirmed by an actual query and UDP/53.

Listing 12: NMAP, ICMP, and DNS tests from PR2.

```
ping -c 2 -W 2 10.1.1.10
--- 10.1.1.10 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss
```

```
ping -c 2 -W 2 203.0.113.10
--- 203.0.113.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
```

```
nmap -n -Pn -p 80,443,22 172.16.10.10
PORT STATE SERVICE
22/tcp filtered ssh
80/tcp open http
443/tcp open https
```

```
nmap -n -Pn -p 25,110,143,80 172.16.10.20
PORT STATE SERVICE
25/tcp open smtp
80/tcp filtered http
110/tcp open pop3
143/tcp open imap
```

```
nmap -n -Pn -p 53 172.16.10.30
PORT STATE SERVICE
53/tcp open domain
```

```
dig @172.16.10.30 exemplo.local +short
```



```
172.16.10.10

nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 172.16.10.30
PORT STATE SERVICE
53/udp open domain
```

To complete requirement 5, the allowed and disallowed protocols from PR1 and PR2 to OUT were also tested directly. The HTTP, HTTPS, and DNS ports appear as **closed**, not **filtered**, because the OUT host received the traffic but did not have those services active. In contrast, SSH and SMTP appear as **filtered**, demonstrating blocking by the firewall.

Listing 13: Direct validation from PR1/PR2 to OUT.

```
PR1 -> OUT
ping -c 2 -W 2 203.0.113.10
2 packets transmitted, 2 received, 0% packet loss

nmap -n -Pn -p 80,443,53,22,25 203.0.113.10
PORT STATE SERVICE
22/tcp filtered ssh
25/tcp filtered smtp
53/tcp closed domain
80/tcp closed http
443/tcp closed https

nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 203.0.113.10
PORT STATE SERVICE
53/udp closed domain

PR2 -> OUT
ping -c 2 -W 2 203.0.113.10
2 packets transmitted, 2 received, 0% packet loss

nmap -n -Pn -p 80,443,53,22,25 203.0.113.10
PORT STATE SERVICE
22/tcp filtered ssh
25/tcp filtered smtp
53/tcp closed domain
80/tcp closed http
443/tcp closed https

nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 203.0.113.10
PORT STATE SERVICE
53/udp closed domain
```

2.1.6 Requirements 3 and 8 - Tests from OUT

The OUT zone cannot access the private zones, but it can access the authorized services in the DMZ. Output 14 shows that access to PR1 failed, while the public DMZ services were reachable according to the specification.

Listing 14: NMAP and ICMP tests from OUT.

```
ping -c 2 -W 2 10.1.1.10
--- 10.1.1.10 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss

ping -c 2 -W 2 172.16.10.10
--- 172.16.10.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
```

```

nmap -n -Pn -p 80,443,22 172.16.10.10
PORT STATE SERVICE
22/tcp filtered ssh
80/tcp open http
443/tcp open https

nmap -n -Pn -p 25,110,143 172.16.10.20
PORT STATE SERVICE
25/tcp open smtp
110/tcp filtered pop3
143/tcp filtered imap

nmap -n -Pn -p 53,25 172.16.10.30
PORT STATE SERVICE
25/tcp filtered smtp
53/tcp open domain

dig @172.16.10.30 exemplo.local +short
172.16.10.10

nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 172.16.10.30
PORT STATE SERVICE
53/udp open domain

```

From OUT, SMTP appeared as **open**, while POP3 and IMAP appeared as **filtered**; thus, the policy exposes only the sending of email. The actual query to **exemplo.local** and the NMAP UDP/53 test complete the demonstration of DNS from OUT to the DMZ.

2.1.7 Requirement 7 - Tests from the DMZ

The DMZ could communicate with OUT only via ICMP, SMTP, and DNS, but it could not access the private zones. Output 15 demonstrates this behavior. The SMTP and DNS ports on OUT appear as **closed**, not as **filtered**; this means that the firewall allowed the attempt, but the OUT host did not have those services open. The HTTP port appears as **filtered**, because it was not part of the DMZ-to-OUT policy.

Listing 15: NMAP and ICMP tests from the DMZ.

```

ping -c 2 -W 2 203.0.113.10
--- 203.0.113.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss

ping -c 2 -W 2 10.1.1.10
--- 10.1.1.10 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss

nmap -n -Pn -p 25,53,80 203.0.113.10
PORT STATE SERVICE
25/tcp closed smtp
53/tcp closed domain
80/tcp filtered http

nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 203.0.113.10
PORT STATE SERVICE
53/udp open|filtered domain

```

The relevant point is that, together with the configured ACL and *policy map*, the result is compatible with allowed UDP/53 traffic. Because in UDP there is not always a clear application response, the **open|filtered** state does not by itself prove that the service replied, but it does not contradict the defined policy. The HTTP port remains **filtered**, demonstrating the blocking

of unauthorized traffic.

Listing 16: Additional DNS validation over UDP.

```
OUT -> DMZ-DNS
dig @172.16.10.30 exemplo.local +short
172.16.10.10

nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 172.16.10.30
PORT STATE SERVICE
53/udp open domain

DMZ-DNS -> OUT
nmap -sU -n -Pn -p 53 --max-retries 1 --host-timeout 15s 203.0.113.10
PORT STATE SERVICE
53/udp open|filtered domain
```

2.1.8 Requirements 9 and 10 - Firewall access only via SSH and denial of all other traffic

The assignment required that the firewall be reachable from all zones, but only via SSH. First, it was confirmed with NMAP that port 22 was open and that Telnet was blocked. Next, a real SSH session was established from PR1 to the firewall. Output 17 shows the authenticated session, including `show users`, where the VTY line appears associated with user `admin` and source `10.1.1.10`.

Listing 17: Validation of access to the firewall only via SSH.

```
nmap -n -Pn -p 22,23 10.1.1.254
PORT STATE SERVICE
22/tcp open ssh
23/tcp filtered telnet

ssh admin@10.1.1.254
Password:

FW-CAMPUS#show clock
*11:41:26.183 UTC Wed May 6 2026

FW-CAMPUS#show users
   Line User Host(s) Idle Location
*  2 vty 0 admin idle 00:00:00 10.1.1.10
```

Output 18 tests ports 22/23 from the four zones: SSH is reachable and Telnet is always **filtered**. The *self zone* policy allows only SSH.

Listing 18: NMAP validation of access to the firewall from all zones.

```
PR1 -> firewall 10.1.1.254
PORT STATE SERVICE
22/tcp open ssh
23/tcp filtered telnet

PR2 -> firewall 10.2.2.254
PORT STATE SERVICE
22/tcp open ssh
23/tcp filtered telnet

DMZ -> firewall 172.16.10.254
PORT STATE SERVICE
22/tcp open ssh
```

```
23/tcp filtered telnet

OUT -> firewall 203.0.113.254
PORT STATE SERVICE
22/tcp open  ssh
23/tcp filtered telnet
```

2.1.9 Comparison with AI proposal

An AI tool was also asked to propose a ZBPF configuration for the campus network policy. The proposal followed the theoretical strategy recommended in the assignment: create ACLs per server, group protocols by service with *class maps match-any*, combine protocols and destinations with *class maps match-all*, and finally apply these classes in the *policy maps*. A representative excerpt of the proposal is presented in Output 19.

Listing 19: Representative excerpt of the AI-generated proposal for the ZBPF policy.

```
ip access-list extended ACL_WEB_SERVER
 permit ip any host 172.16.10.10

class-map type inspect match-any CM_WEB_PROTOCOLS
 match protocol http
 match protocol https

class-map type inspect match-all CM_OUT_TO_WEB
 match access-group name ACL_WEB_SERVER
 match class-map CM_WEB_PROTOCOLS

policy-map type inspect PM_OUT_TO_DMZ
 class type inspect CM_OUT_TO_WEB
  inspect
 class class-default
  drop log
```

The AI-generated proposal was useful as a starting point, mainly because it suggested the separation between destination ACLs, protocol classes, and nested *class maps*. However, the presented proposal was only partial: it covered access to the Web server in the DMZ, but it did not fully solve all the requirements of the campus policy. In particular, it did not fully include PR1–PR2 isolation, NAT *overload* from the private zones to OUT, the DMZ→OUT and OUT→DMZ policies for Email and DNS, or the *self zone* policy to allow only SSH. Therefore, the final configuration had to be more extensive and more specific than the AI proposal. The tests with NMAP, ICMP, DNS, and SSH confirmed that the implemented configuration fulfilled the complete policy, while the AI proposal only correctly captured the modular strategy.

2.2 Exercise 6 - Defense against DoS attacks

Exercise 6 reuses the topology and the final ZBPF policy from the previous exercise in the lab sheet, allowing only HTTP and ICMP traffic from the OUTSIDE zone to the INSIDE server. For this reason, the complete details of that configuration are not included in the main body of the report, which uses it only as a starting point for the DoS tests.

2.2.1 Item 1 - Base policy with HTTP and ICMP classes

The DoS exercise reused the two-zone topology. The OUTSIDE zone contained the attacker, the INSIDE zone contained the server, and the ZBPF firewall allowed only HTTP and ICMP to server 222.10.10.1.

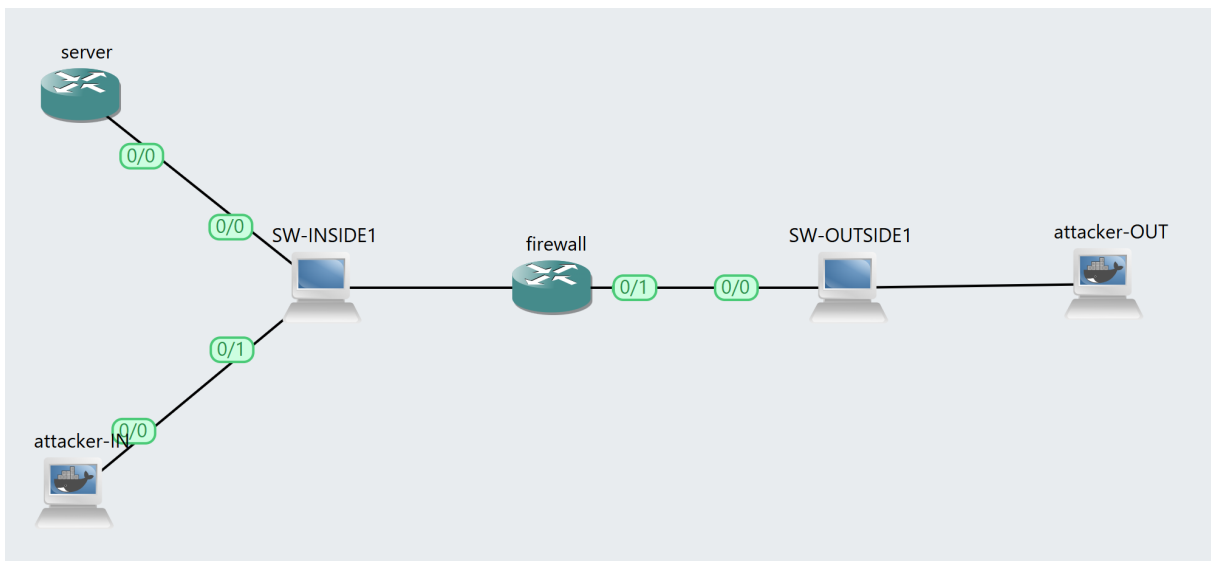


Figure 26: Topology used in the analysis of ICMP flood and TCP SYN flood attacks (in GNS3 accessed through the browser, the switch icons appear with the default PC icon, even though they are switches).

The base policy was split into two classes, one for ICMP and one for HTTP, as defined in the assignment. This made it possible to observe the inspection counters separately and to apply *policing* independently.

Listing 20: Base ZBPF policy for the DoS exercise.

```
ip access-list extended TO_IN_HOST
 permit ip any host 222.10.10.1

class-map type inspect match-all ICMP_CLASSMAP
 match access-group name TO_IN_HOST
 match protocol icmp

class-map type inspect match-all WWW_CLASSMAP
 match access-group name TO_IN_HOST
 match protocol http

policy-map type inspect DOS_POLICYMAP
 class type inspect WWW_CLASSMAP
  inspect
 class type inspect ICMP_CLASSMAP
  inspect
 class class-default
  drop log
```

2.2.2 Item 2 - TCP SYN flood without source randomization

The first attack was executed with `hping3 -i u10000 -S -p 80 222.10.10.1`. Because the source was the attacker's real address, the server responded with `SYN/ACK` and the attacking system responded with `RST`. Capture 21 shows this sequence: `SYN`, `SYN/ACK`, and `RST`. Therefore, the sessions did not remain *half-open* indefinitely.

Listing 21: Summary of the Wireshark capture of the SYN flood with a fixed source.

```
Total packets parsed: 1630
First TCP packets:
TCP 111.10.10.2:2463->222.10.10.1:80 flags=S
TCP 111.10.10.2:2464->222.10.10.1:80 flags=S
TCP 222.10.10.1:80->111.10.10.2:2463 flags=SA
TCP 222.10.10.1:80->111.10.10.2:2464 flags=SA
TCP 111.10.10.2:2463->222.10.10.1:80 flags=R
TCP 111.10.10.2:2464->222.10.10.1:80 flags=R
```

Output 22 confirms that the attacker received almost all the responses. Thus, this test did not significantly exhaust the server's sessions.

Listing 22: Output of the TCP SYN attack with a fixed source.

```
hping3 -i u10000 -S -p 80 222.10.10.1

--- 222.10.10.1 hping statistic ---
543 packets transmitted, 537 packets received, 2% packet loss
```

2.2.3 Item 3 - TCP SYN flood with source randomization

In the second test, `-rand-source` was used. In this case, the server sent `SYN/ACK` packets to spoofed addresses, which did not complete the *three-way handshake*. Capture 23 shows several random sources sending `SYN` to `222.10.10.1:80`, followed by `SYN/ACK` responses from the server to those sources.

Listing 23: Summary of the Wireshark capture of the SYN flood with a random source.

```
Total packets parsed: 1477
First TCP packets:
TCP 86.100.181.4:2927->222.10.10.1:80 flags=S
TCP 6.215.101.145:2928->222.10.10.1:80 flags=S
TCP 222.10.10.1:80->86.100.181.4:2927 flags=SA
TCP 187.174.21.204:2929->222.10.10.1:80 flags=S
TCP 222.10.10.1:80->6.215.101.145:2928 flags=SA
TCP 21.101.102.101:2930->222.10.10.1:80 flags=S
TCP 222.10.10.1:80->187.174.21.204:2929 flags=SA
TCP 222.10.10.1:80->21.101.102.101:2930 flags=SA
```

Output 24 shows the effect on the firewall: hundreds of *half-open* sessions were created. The counter `Current session counts (estab/half-open/terminating) [0:535:0]` demonstrates that the attack succeeded before mitigation, because the inspection table accumulated incomplete sessions.

Listing 24: ZBPF counters after SYN flood with a random source.

```
Class-map: WWW_CLASSMAP (match-all)
Inspect
  tcp packets: [0:3784]
  Session creations since subsystem startup or last reset 1618
```

```
Current session counts (estab/half-open/terminating) [0:535:0]
Maxever session counts (estab/half-open/terminating) [0:535:0]
Last session creation rate 1078
Last half-open session total 535
```

2.2.4 Item 4 - Mitigation with rate limiting

The first countermeasure was the introduction of `police rate 128000 burst 8000` in the HTTP and ICMP classes. The *policer* transmits packets within the configured profile and discards packets that exceed that profile. Output 25 shows the form of the applied policy.

Listing 25: ZBPF policy with rate limiting.

```
policy-map type inspect DOS_POLICYMAP
class type inspect WWW_CLASSMAP
inspect
  police rate 128000 burst 8000
class type inspect ICMP_CLASSMAP
inspect
  police rate 128000 burst 8000
class class-default
drop log
```

In the *ICMP flood* attack, the Wireshark capture showed a rapid sequence of ICMP Echo Request packets sent by 111.10.10.2 to 222.10.10.1. The output of `hping3` in *flood* mode is not used as the main evidence of success or failure, because in this mode the tool prioritizes continuous packet transmission. The mitigation is demonstrated mainly by the ZBPF IOS counters: packets within the profile were transmitted and excess packets were discarded by the firewall.

Listing 26: Representative summary of the Wireshark capture of the ICMP flood.

```
hping3 -1 --flood 222.10.10.1

ICMP 111.10.10.2 -> 222.10.10.1 Echo request
ICMP 111.10.10.2 -> 222.10.10.1 Echo request
ICMP 111.10.10.2 -> 222.10.10.1 Echo request
ICMP 111.10.10.2 -> 222.10.10.1 Echo request

--- 222.10.10.1 hping statistic ---
3182850 packets transmitted, 0 packets received, 100% packet loss
```

The *policing* counters show a clear difference between conforming and exceeded packets. Output 27 confirms that the firewall dropped 42132 ICMP packets because they exceeded the configured policy.

Listing 27: Policing counters after ICMP flood.

```
Class-map: ICMP_CLASSMAP (match-all)
Inspect
  icmp packets: [0:2252]
Police
  rate 128000 bps,8000 limit
  conformed 2252 packets, 97860 bytes; actions: transmit
  exceeded 42132 packets, 1777122 bytes; actions: drop
```

In the *TCP SYN flood* attack, the same mechanism limited the number of accepted HTTP packets. Output 28 shows that 242 packets were dropped for exceeding the profile. However, *policing* alone does not eliminate the problem of *half-open* TCP sessions: the counter still shows

[0:1795:0]. Thus, this countermeasure reduces the incoming rate and the pressure on the server, but the specific mitigation of the incomplete TCP state is demonstrated only in the next item, with the *parameter map*.

Listing 28: Policing counters after TCP SYN flood.

```
Class-map: WWW_CLASSMAP (match-all)
  Inspect
    tcp packets: [44:1509]
    Current session counts (estab/half-open/terminating) [0:1795:0]
  Police
    rate 128000 bps,8000 limit
    conformed 1553 packets, 83862 bytes; actions: transmit
    exceeded 242 packets, 13068 bytes; actions: drop
```

Different values of `rate` and `burst` were also tested. With 64000 bps and `burst 4000`, legitimate traffic experienced greater loss because the profile was more restrictive. With 256000 bps and `burst 16000`, the firewall transmitted more packets and recorded fewer exceeded packets. Thus, the `rate` parameter controls the allowed average rate, while `burst` defines tolerance for short bursts. A value that is too low mitigates the flood better, but it also increases the probability of affecting legitimate traffic.

Listing 29: Comparison of policing parameters.

```
rate 64000 burst 4000:
1000 packets transmitted, 159 packets received, 85% packet loss
police counters: conformed packets present; exceeded packets increased

rate 256000 burst 16000:
1000 packets transmitted, 241 packets received, 76% packet loss
police counters: conformed 1316 packets; exceeded 0 packets
```

2.2.5 Mitigation of half-open sessions with a parameter map

For the specific case of a *TCP SYN flood*, the second countermeasure was to use a *parameter map* with `tcp synwait-time 3`, according to the documented approach for ZBPF in Cisco IOS [4]. With this configuration, ZBPF can intervene earlier when a TCP session does not complete the *handshake*. The HTTP policy started using `inspect PARMAP`.

Listing 30: Parameter map used to reduce the waiting time for completion of the TCP three-way handshake.

```
parameter-map type inspect PARMAP
  tcp synwait-time 3

policy-map type inspect DOS_POLICYMAP
  class type inspect WWW_CLASSMAP
    inspect PARMAP
    police rate 128000 burst 8000
  class type inspect ICMP_CLASSMAP
    inspect
    police rate 128000 burst 8000
  class class-default
    drop log
```

Capture 31 is the main evidence of this mitigation. After the *parameter map* was configured, RST packets appear on the link between the firewall and the server. These RST packets have as their source the spoofed addresses used in the attack and the server as their destination. This shows

that the firewall took the initiative to close incomplete sessions on the server side, preventing them from remaining accumulated for too long.

Listing 31: Summary of the Wireshark capture after applying the parameter map.

```
Total packets parsed: 4210
Flag totals:
R: 986
S: 2094
SA: 69

First TCP packets:
TCP 249.22.71.82:1231->222.10.10.1:80 flags=S
TCP 93.6.124.131:1232->222.10.10.1:80 flags=S
TCP 222.10.10.1:80->93.6.124.131:1232 flags=SA

RST samples:
TCP 249.22.71.82:1231->222.10.10.1:80 flags=R
TCP 93.6.124.131:1232->222.10.10.1:80 flags=R
TCP 221.151.198.242:1233->222.10.10.1:80 flags=R
TCP 10.113.213.107:1234->222.10.10.1:80 flags=R
```

Output 32 confirms the operational effect: although the maximum number of *half-open* sessions reached 3201 during the attack, the current counter returned to zero. This demonstrates that ZBPF no longer only limited the rate, but also began clearing incomplete TCP sessions.

Listing 32: ZBPF counters after SYN flood with a parameter map.

```
Class-map: WWW_CLASSMAP (match-all)
  Inspect
    tcp packets: [65:2030]
    Session creations since subsystem startup or last reset 3345
    Current session counts (estab/half-open/terminating) [0:0:0]
    Maxever session counts (estab/half-open/terminating) [0:3201:0]
    Last half-open session total 0
  Police
    rate 128000 bps,8000 limit
    conformed 2095 packets, 113130 bytes; actions: transmit
    exceeded 1117 packets, 60318 bytes; actions: drop
```

2.3 Technical discussion

The tests confirm that the ZBPF applied in the campus network operated according to the specification. PR1 and PR2 were isolated, the private zones were not accessible from other zones, the DMZ exposed only the defined services, and the firewall accepted only SSH in the *self zone*. The distinction between NMAP states was essential: **open** confirmed that a service was reachable, **filtered** indicated blocking by the firewall, and **closed** indicated that the firewall allowed the traffic but the service was not active at the destination.

In the defense against DoS, the *TCP SYN flood* with a fixed source was not effective because the real attacker received the **SYN/ACK** packets and responded with **RST**. With a random source, the attack became effective because the server's responses were sent to spoofed addresses and the *handshake* did not complete. ZBPF mitigated the problem in two layers: first, it reduced the rate of accepted packets with *policing*; then it reduced the lifetime of *half-open* sessions with `tcp synwait-time 3`, generating **RST** packets to free server resources.

2.4 Conclusion

The report demonstrated that the ZBPF policy of the campus network meets the ten defined requirements: isolation between PR1 and PR2, NAT *overload* for the private zones, controlled exposure of the DMZ, firewall access only via SSH, and denial of unauthorized traffic. In the DoS component, the Wireshark captures and IOS counters made it possible to distinguish the SYN flood with a real source from the attack with a spoofed source. *Policing* reduced the packet arrival rate, while the *parameter map* with `tcp synwait-time 3` reduced the lifetime of *half-open* TCP sessions, allowing the firewall to free resources on the attacked server.